THE UNIVERSITY OF MELBOURNE

COMP30023 – Computer Systems

Connection oriented transport: Transmission Control Protocol

Dr Lachlan Andrew

© University of Melbourne 2025

Recap – Transport Layer

- Provides services to the Application (or Session) layer using services from the Network layer.
- The Transport layer entities (the hardware or software which actually does the work e.g. OS kernel, processes, NIC) can exist in multiple locations.
- Services provide “logical” communication channels between processes running on different hosts:
  - Connectionless: data transfer (UDP)
    - Like text messages
  - Connection-oriented
    - = Connection establishment, data transfer, connection release (TCP)
    - Like phone call

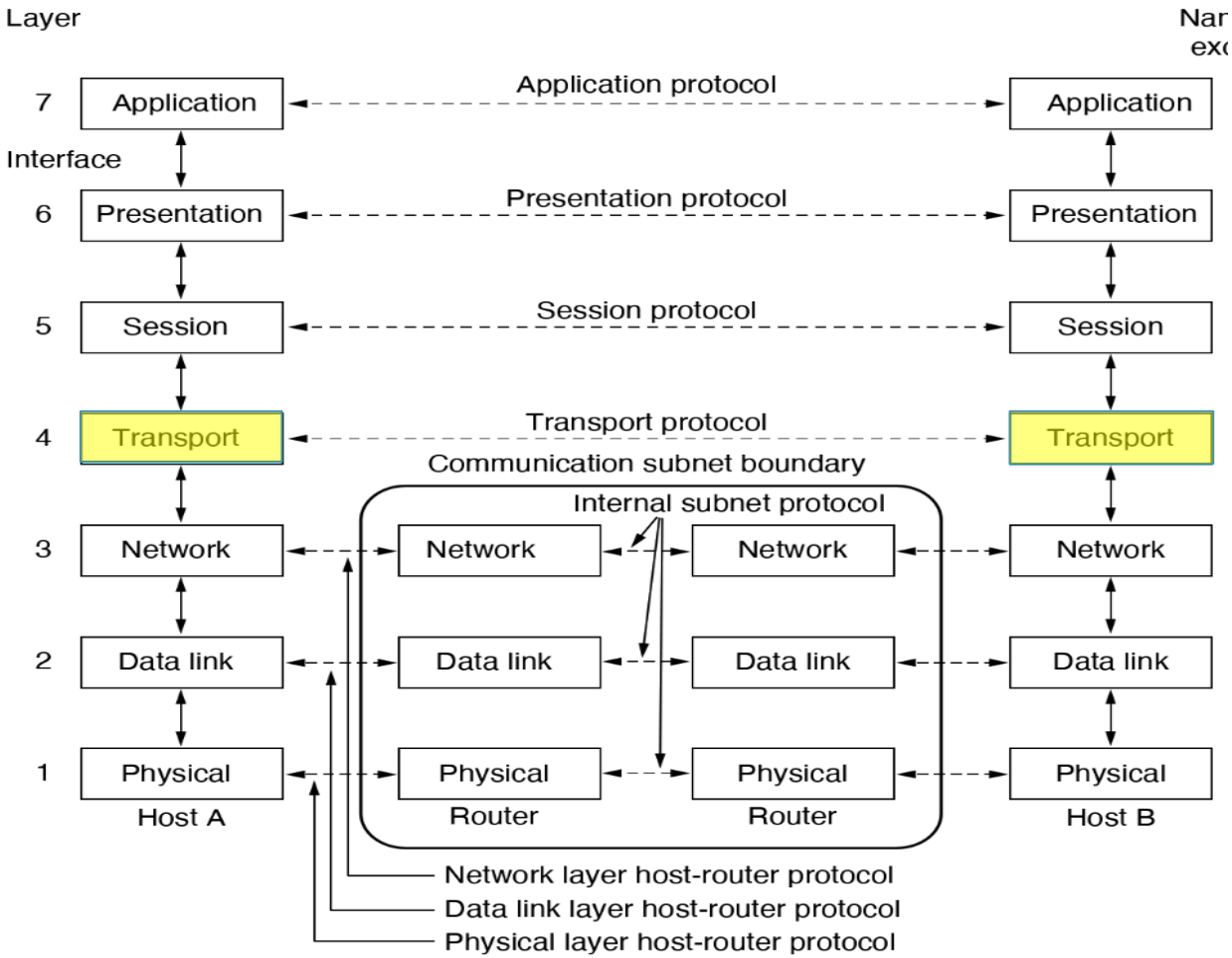
Transmission Control Protocol, TCP

```
    // #define required for VSCode
#define _POSIX_C_SOURCE 200112L
#include <netdb.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

int connfd = 0, re = 1, s, n;
struct addrinfo hints, *res, *rp;

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_INET6;
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_ACTVE;
```

```
s = getaddrinfo("localhost",
                "5000", &hints, &res);
for (rp = res; rp != NULL;
            rp = rp->ai_next) {
    connfd = socket(rp->ai_family,
                    rp->ai_socktype,
                    rp->ai_protocol);
    if (connfd == -1) continue;
    if (connect(connfd,
                rp->ai_addr,
                rp->ai_addrlen) != -1
        break;
    close(connfd);
}
write (connfd, "Hello, network!\r\n");
close (connfd);
```

© 2025 University of Melbourne

Position in the stack: Network layer



- Use data
- Tidy up e2e
- Get data e2e
- Tidy up p2p
- Get data p2p

Summary

- Connection oriented transport protocols
- Introduction to Transmission Control Protocol (TCP)
  - Connecting
  - Sending
  - Closing

Transport layer – layer 4

Network layer (below us) gets packets end-to-end, but

- Packets may be lost
- Packets may arrive out of order
- Packets may be duplicated
- Packets may arrive faster than we can process them
- Packets don’t say which application they are for
- Packets may be corrupted

The transport layer provides a “cleaner”, more “friendly” set of services

©

TCP - Overview

- The Transmission Control Protocol lets applications transmit and receive a stream of bytes, without worrying about
  - Segmenting into IP datagrams (it is “stream oriented”)
  - Bytes being dropped, or duplicated (it is “reliable”)
  - Bytes arriving out of order (it is “in order”)
- TCP transport entity manages TCP streams and interfaces to the IP layer
  - TCP entity accepts user data streams, and segments them into pieces <64Kb (often 1460 bytes in order to fit the IP and TCP headers into a single Ethernet frame), and sends each piece as a separate IP datagram
- Recipient TCP entities reconstruct the original byte streams from the encapsulation

Transport/TCP service primitives

- Primitives: core functions which allow interface with transport services (in particular TCP)

Primitive	Packet Sent	Meaning
LISTEN	(none)	Block until something tries to connect
CONNECT	CONNECTION REQ	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until DATA packet arrives
DISCONNECT	DISCONNECTION REQ	This sides wants to release the connection

"Select" is a non-TCP primitive allowing non-blocking receive

Exercise: Identify primitives in this code and the first slide

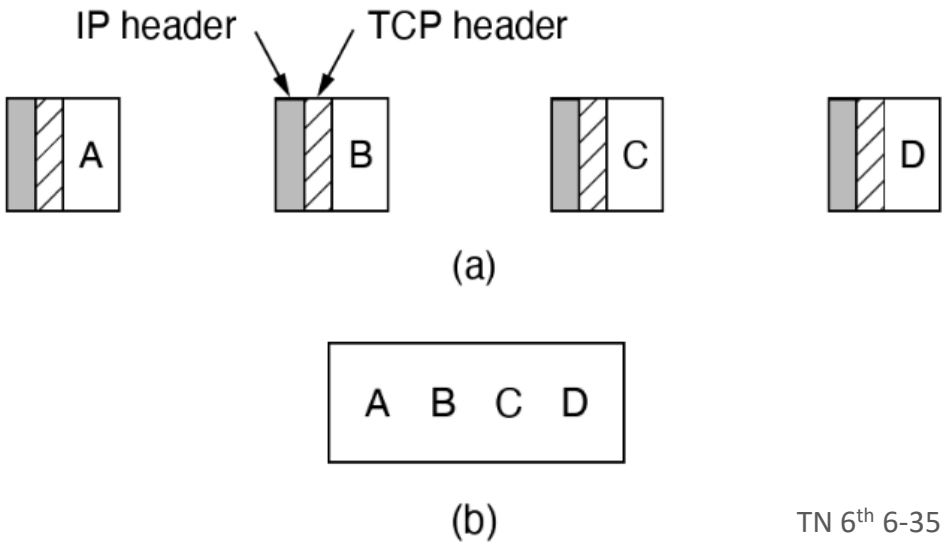
```
    // #define required for VSCode
#define _POSIX_C_SOURCE 200112L
#include <netdb.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

int re = 1, s, n;
char recvBuff[1024];

struct sockaddr_in address;
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);


Int listenfd = socket (AF_INET,
                       SOCK_STREAM, 0);
```

```
setsockopt(listenfd, SOL_SOCKET,
                SO_REUSEADDR,
                &re, sizeof(re));
bind(listenfd, (struct sockaddr*)&address,
        sizeof(address));
// 10 is max num of client connections  to queue
listen(listenfd, 10);
struct sockaddr_storage client_addr;
socklen_t client_addr_size =sizeof client_addr;
int connfd = accept(listenfd,
            (struct sockaddr*)&client_addr,
             &client_addr_size);
read (connfd, recvBuff, sizeof (recvBuff));
write (connfd, “Hello, back!\r\n”);
close (connfd);
```

© 2025 University of Melbourne

TCP – Service Model

- Example:
  - (a) Four 512-byte segments sent as separate IP datagrams

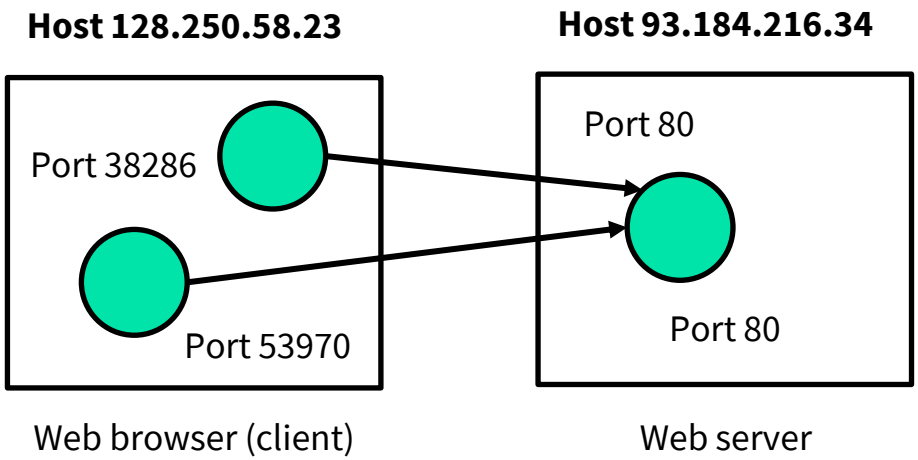


TN 6th 6-35

  - (b) The 2048 bytes of data (may be) delivered to the application in a single READ call

TCP – Service Model

- The sender and receiver both create sockets
  - A kernel data structure, named by the *5-tuple* of IP address and port number of local and remote hosts, and the protocol
  - For TCP service to be activated, connections must be explicitly established between a socket at a sending host (src-host, src-port) and a socket at a receiving host (dest-host, dest-port)
- Example:
  - 2 sockets on port 80



Features of TCP connections

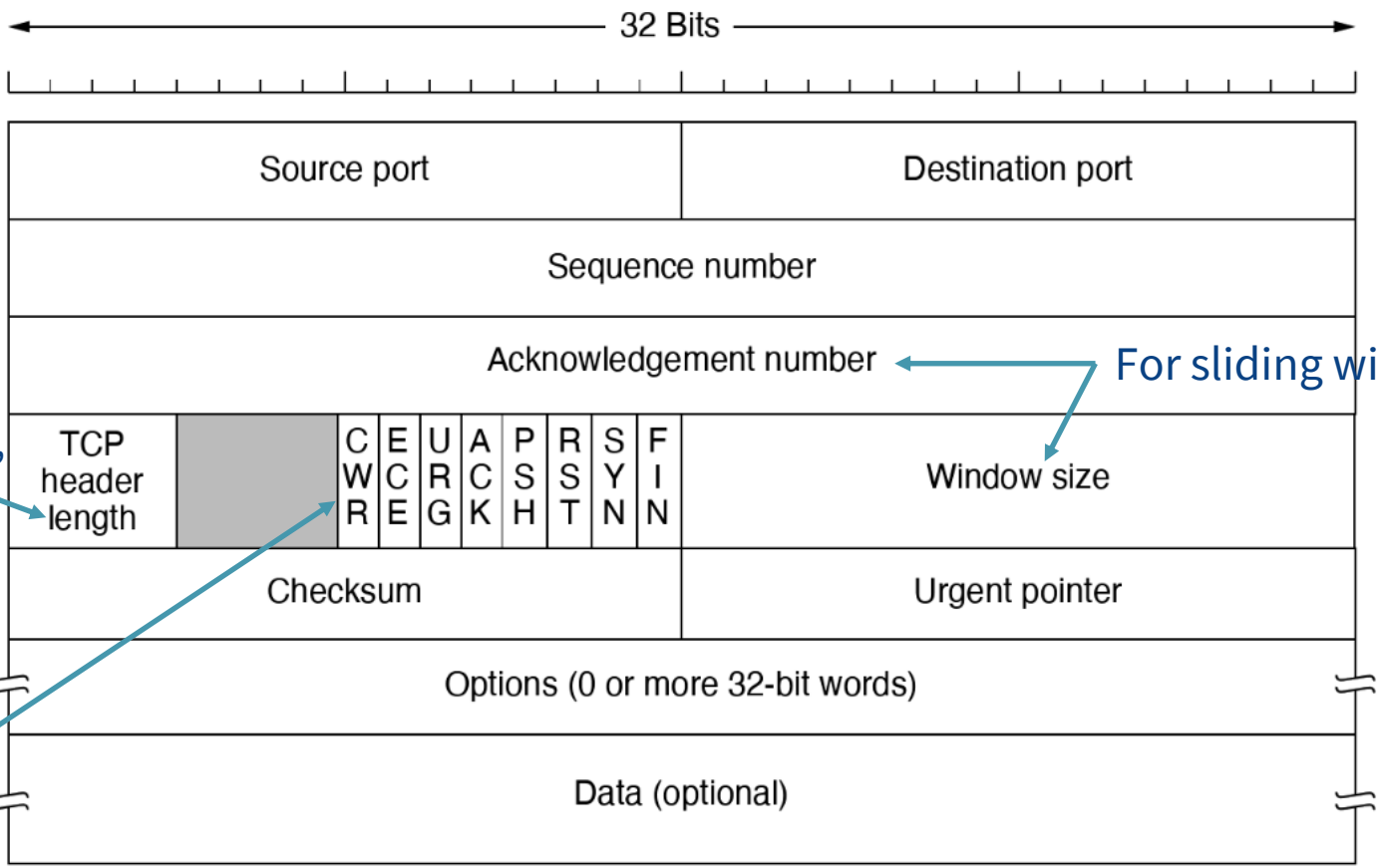
- TCP connections are:
  - Full duplex - data in both directions simultaneously
  - End to end - exact pairs of senders and receivers
  - Byte streams, not message streams
    - Application layer message boundaries are not preserved
    - Can't tell where one `write()` ends and the next starts
  - Buffer capable
    - TCP entity can choose to buffer prior to sending or not
    - Buffering reduces overhead (fewer headers), but increases delay

TCP Properties

- Data is exchanged between TCP entities in segments
  - each has a 20– 60 byte header, plus *zero or more* data bytes
- TCP entities decide how large segments should be, given two constraints:
  - IP payload < 65,515 byte
  - Maximum Transfer Unit (MTU) - generally 1500 bytes
- Sliding window protocol
  - Initial use: reliable data delivery without overloading the receiver
  - Now also tied closely with congestion control
  - Described next lecture

TCP – Header

TN 6th 6-36



For sliding window

32-bit words, not bytes

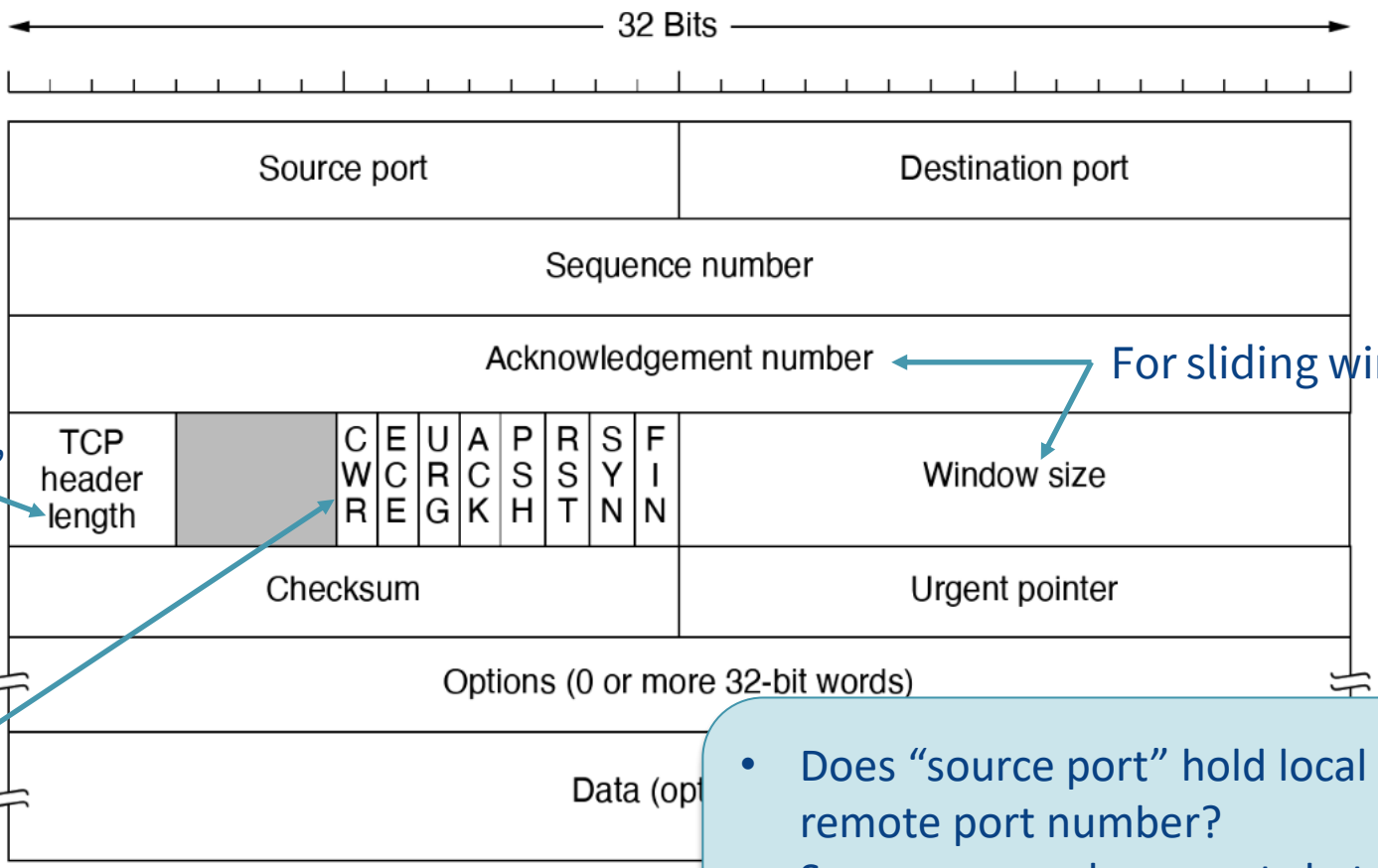
Week 12 revision: What do all the flags do?

TCP – Important Header fields

Name	Description
Source port	Sending port
Destination port	Receiving port
Sequence Number	If **SYN**=1: initial sequence number if **SYN**=0: is accumulated sequence number of the first data byte of this segment
Acknowledgement number	If **ACK**=1: next sequence number that the sender of the **ACK** is expecting
Data offset	Size of the TCP Header (20-60 bytes = 5-15 x 32-bit)
Flags	Single bit flags (**SYN, ACK, RST, FIN**, etc.)
Window size	Size of receive window – how much data the sender of this segment is willing to receive

TCP – Header

TN 6th 6-36



For sliding window

32-bit words, not bytes

Week 12 revision: What do all the flags do?

- Does "source port" hold local or remote port number?
- Sequence number counts bytes, not packets; why?
- What happens if I transmit 5GB?

© 2025 University of Melbourne

Connection establishment

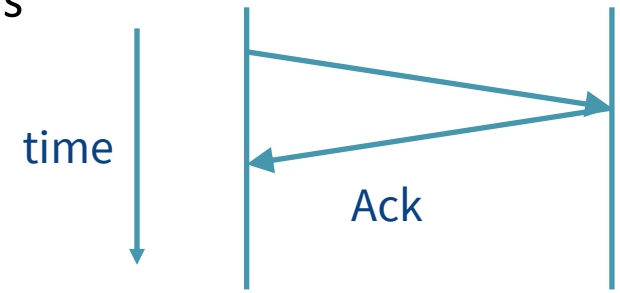
```
    // #define required for VSCode
#define _POSIX_C_SOURCE 200112L
#include <netdb.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

int connfd = 0, re = 1, s, n;
struct addrinfo hints, *res, *rp;

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_INET6;
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_ACTVE;
```

```
s = getaddrinfo("localhost",
                "5000", &hints, &res);
for (rp = res; rp != NULL;
            rp = rp->ai_next) {
    connfd = socket(rp->ai_family,
                    rp->ai_socktype,
                    rp->ai_protocol);
    if (connfd == -1) continue;
    if (connect(connfd,
                rp->ai_addr,
                rp->ai_addrlen) != -1
        break;
    close(connfd);
}
write (connfd, "Hello, network!\r\n");
close (connfd);
```

© 2025 University of Melbourne

Connection establishment issues

- TCP is a *connection orientated* protocol running over a *connectionless* network layer (IP)
- When networks can lose, store and duplicate packets, connection establishment can be complicated
  - congested networks may delay acknowledgements
  - incurring repeated multiple transmissions
  - any of which may not arrive at all or out of sequence – delayed duplicates

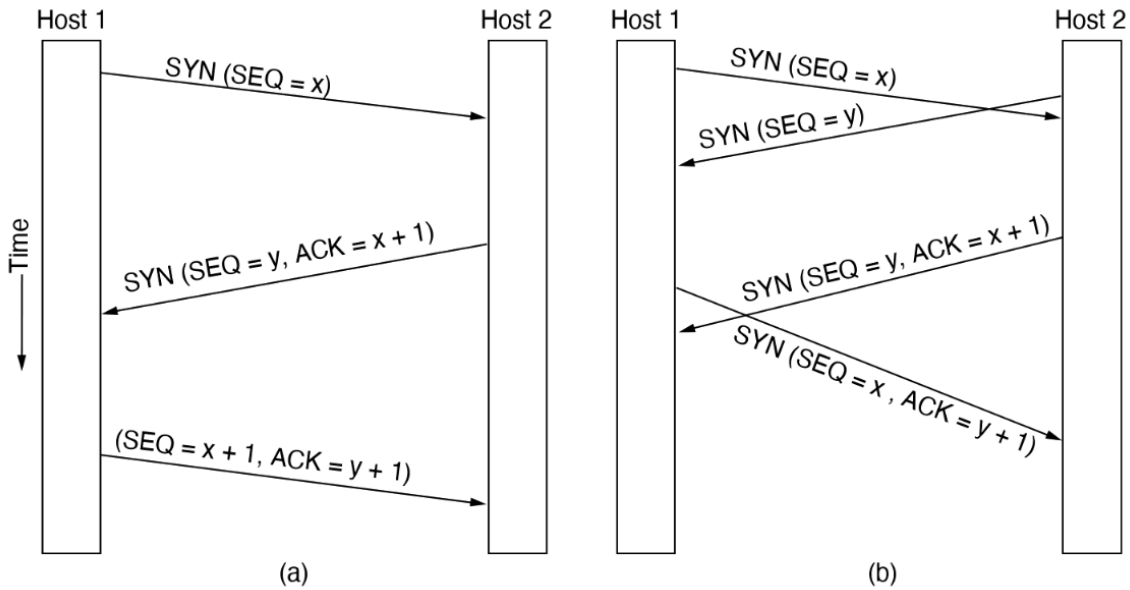


Three-way handshake

- Goals of reliable connection establishment:
  - Ensure one (and only one) connection is established, even if some set-up packets get lost and/or retransmitted
  - Establish initial sequence numbers for sliding window

- Three-way handshake:
  - A proposed solution, which avoids problems that can occur when both sides allocate same sequence numbers by accident (e.g. after host/router crash) (cf. Tomlinson, 1975).
  - Sender and receivers exchange information about which sequencing strategy each will use, and agree on it before transmitting segments

Three-way handshake



TN 6th 6-37

Normal operation Simultaneous connection attempts.

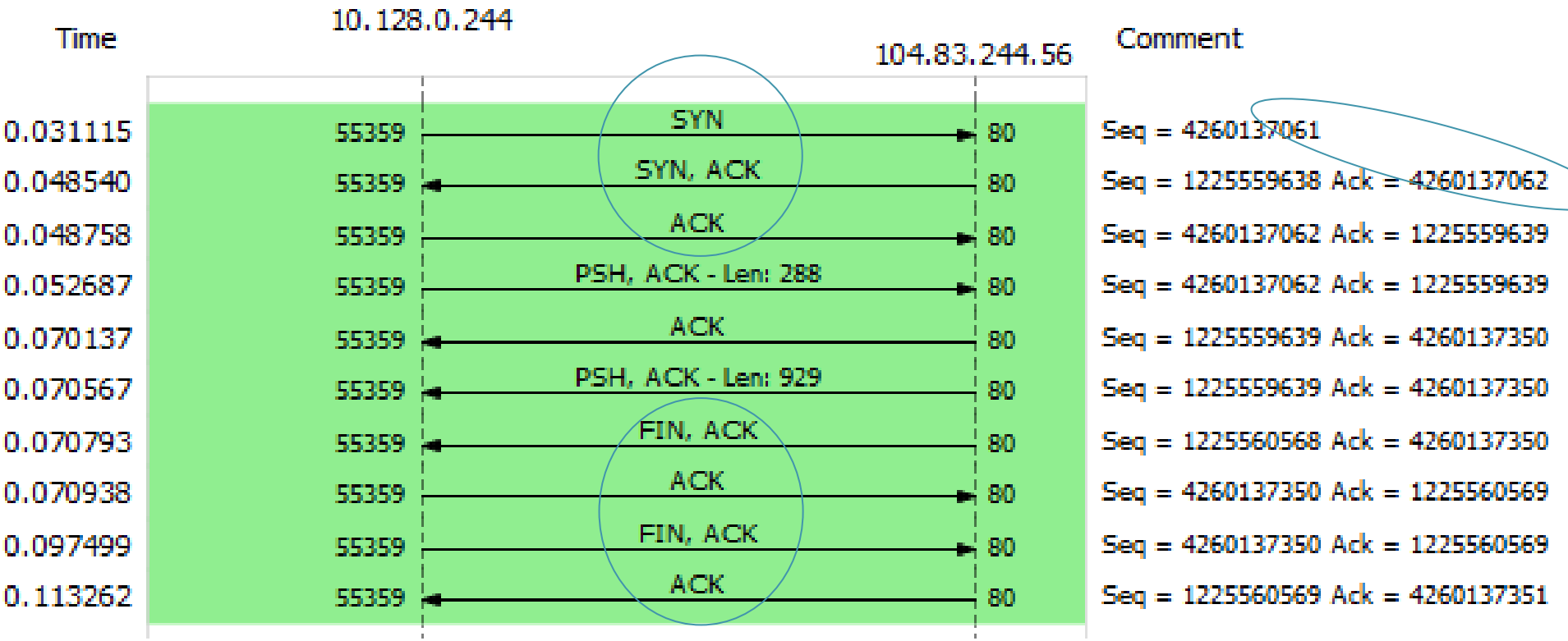
- Two simultaneous connection attempts results in only one connection (uniquely identified by end points).

- At end, Host 1 and Host 2 have agreed on respective sequence numbers

Synchronisation

- SYN is used for synchronization during connection establishment
  - SYN and FIN each count for 1 byte of sequence number
  - ➢ The sequence number of the first data byte is (seq num of SYN)+1
- Sequence Number – first byte of this segment's payload
  - Offset by a random number – initial value is arbitrary, offset will be reflected in both Sequence and Acknowledgement numbers
- Acknowledgement Number – *next* byte the sender expects to receive
  - Bytes received without gaps – a missing segment will stop this incrementing, even if later segments have been received

Wireshark Example



Wireshark Example

Time	10.128.0.244		208.67.222.222	104.83.244.56	Comment
0.000000	65161	Standard query 0xb98b A static.b... →	53		DNS: Standard query 0xb98b A static.bbci.co.uk
0.016581	65161	← Standard query response 0xb98b ...	53		DNS: Standard query response 0xb98b A static.bbc...
0.031115	55359	55359 → 80 [SYN] Seq=4260137061 Win=17520 Len=0 MSS=1460... →		80	TCP: 55359 → 80 [SYN] Seq=4260137061 Win=1...
0.048540	55359	← 80 → 55359 [SYN, ACK] Seq=1225559638 Ack=4260137062 Win=...		80	TCP: 80 → 55359 [SYN, ACK] Seq=1225559638 ...
0.048758	55359	55359 → 80 [ACK] Seq=4260137062 Ack=1225559639 Win=17408 ... →		80	TCP: 55359 → 80 [ACK] Seq=4260137062 Ack=1...
0.052687	55359	GET /weather/0.5.284/images/icons/individual_56_icons/en_on_light... →		80	HTTP: GET /weather/0.5.284/images/icons/individ...
0.070137	55359	← 80 → 55359 [ACK] Seq=1225559639 Ack=4260137350 Win=30272 ...		80	TCP: 80 → 55359 [ACK] Seq=1225559639 Ack=4...
0.070567	55359	← HTTP/1.1 200 OK (GIF89a)		80	HTTP: HTTP/1.1 200 OK (GIF89a)
0.070793	55359	← 80 → 55359 [FIN, ACK] Seq=1225560568 Ack=4260137350 Win=3...		80	TCP: 80 → 55359 [FIN, ACK] Seq=1225560568 ...
0.070938	55359	55359 → 80 [ACK] Seq=4260137350 Ack=1225560569 Win=16384 ... →		80	TCP: 55359 → 80 [ACK] Seq=4260137350 Ack=1...
0.097499	55359	55359 → 80 [FIN, ACK] Seq=4260137350 Ack=1225560569 Win=1... →		80	TCP: 55359 → 80 [FIN, ACK] Seq=4260137350 ...
0.113262	55359	← 80 → 55359 [ACK] Seq=1225560569 Ack=4260137351 Win=30272 ...		80	TCP: 80 → 55359 [ACK] Seq=1225560569 Ack=4...

TCP Synchronization Recap

- SYN bit is used to establish a connection
  - Connection request has SYN=1, ACK=0
  - Connection reply has SYN=1, ACK=1
- SYN is used in both CONNECTION_REQUEST and CONNECTION_ACCEPTED, ACK bit distinguishes between the two
- After connection setup:
  - Sequence Number –first byte of this segment payload (1 + data sent prior to this segment)
    - Offset by a random number – initial value is arbitrary, offset will be reflected in both Sequence and Acknowledgement numbers
  - Acknowledgement Number – next byte the sender expects to receive (data successfully received + 1)
    - Bytes received without gaps – a missing segment will stop this incrementing, even if later segments have been received

TCP Closing

```
    // #define required for VSCode
#define _POSIX_C_SOURCE 200112L
#include <netdb.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

int connfd = 0, re = 1, s, n;
struct addrinfo hints, *res, *rp;

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_INET6;
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_ACTVE;
```

```
s = getaddrinfo("localhost",
                "5000", &hints, &res);
for (rp = res; rp != NULL;
            rp = rp->ai_next) {
    connfd = socket(rp->ai_family,
                    rp->ai_socktype,
                    rp->ai_protocol);
    if (connfd == -1) continue;
    if (connect(connfd,
                rp->ai_addr,
                rp->ai_addrlen) != -1
        break;
    close(connfd);
}
write (connfd, "Hello, network!\r\n");
close (connfd);
```

© 2025 University of Melbourne

TCP Closing

- The FIN flag is used to signify a request to close a connection
- Each FIN is directional: once acknowledged, no new data can be *sent* from the sender to the receiver
  - Data can continue to flow in the other direction
  - E.g. client could send FIN after making request, but before receiving the response
  - Sender of FIN will still retransmit unacknowledged segments
- Typically requires 4 segments to close, 1 FIN and 1 ACK for each direction
  - Can be optimised (FIN, FIN/ACK, ACK) if no unfinished business

TCP Closing

- The RST flag is used to signify a “hard” close of a connection
  - Basically states the sender is closing the connection and will not listen for any further messages
  - Sent in reply to a packet sent to a 5-tuple with no open connection
    - e.g., to invalid data being sent or a crashed process that left a remote socket open, that the OS is now cleaning up
- Can be used to close a connection, but FIN is greatly preferred because it is an orderly shutdown of the connection, as opposed to a reset

And finally…

- SYN Flooding
  - Popular attack in the 90's to denial of service a server
  - Remember back to the arbitrary (random) initial Sequence number
  - This requires the server to remember an initial Sequence number for each received SYN request
  - An attacker would make initial SYN requests then not send the appropriate ACK, causing the server to gradually fill up its queue with sequence numbers for now defunct connections
  - One solution was SYN Cookies
    - Rather than store the sequence number it is derived from connection information and a timer that creates a stateless SYN queue using cryptographic hashing
    - Incurs performance cost in validating SYN Cookies, but preferable to being unresponsive – typically only enabled when under attack

Acknowledgement

- The slides are based on slides prepared by Chris Culnane based on material developed previously by: Michael Kirley, Zoltan Somogyi, Rao Kotagiri, James Bailey and Chris Leckie.
- Some of the images included in the notes were supplied as part of the teaching resources accompanying the text books listed in lecture 1.
  - (And also) Computer Networks, 6th Edition, Tanenbaum A., Wetherall. D. https://ebookcentral.proquest.com/lib/unimelb/detail.action?docID=6481879
- Textbook Reference: Sections 3.1,3.2, 3.5, pp.314-319

Sending and receiving data

```
    // #define required for VSCode
#define _POSIX_C_SOURCE 200112L
#include <netdb.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

int connfd = 0, re = 1, s, n;
struct addrinfo hints, *res, *rp;

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_INET6;
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_ACTVE;
```

```
s = getaddrinfo("localhost",
                "5000", &hints, &res);
for (rp = res; rp != NULL;
            rp = rp->ai_next) {
    connfd = socket(rp->ai_family,
                    rp->ai_socktype,
                    rp->ai_protocol);
    if (connfd == -1) continue;
    if (connect(connfd,
                rp->ai_addr,
                rp->ai_addrlen) != -1
        break;
    close(connfd);
}
write (connfd, "Hello, network!\r\n");
close (connfd);
```

© 2025 University of Melbourne

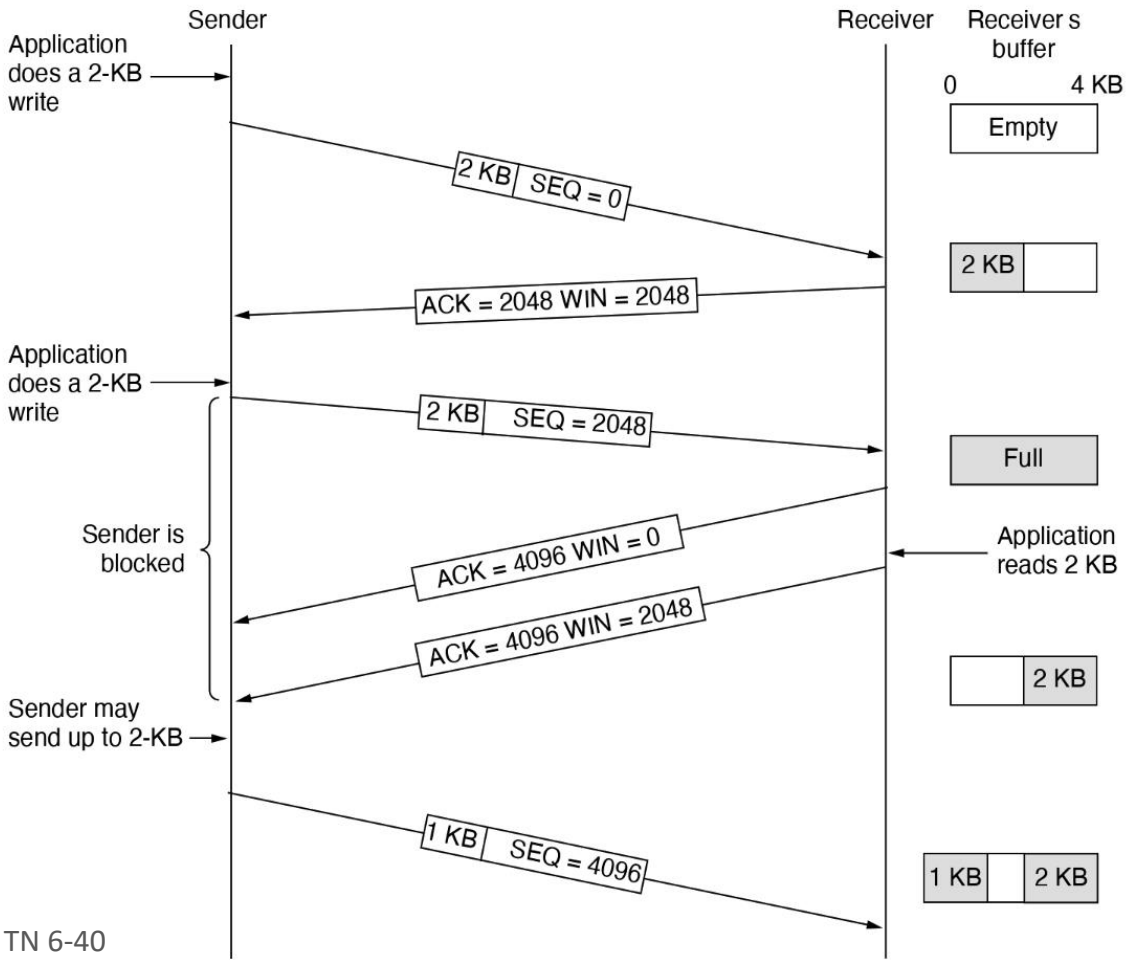
TCP Sliding Window

- Sliding window provides
  - Reliable delivery
  - Flow control (not sending too much to the receiver)
  - In-order delivery

- Sliding window is controlled by receiver
- Determines amount of data the receiver is able to accept
  - Sender and receiver maintain buffers to send and receive data independently of the application
  - No guarantee that data is immediately sent or read from the respective buffers

© 2025 University of Melbourne

TCP Sliding Window



TN 6-40

TCP Sliding Window

- When the window is 0 the sender should not send any data
  - Can send URGENT data
  - Can send “zero window probe”: 0 byte segment that causes the receiver to re-announce the next expected byte and window size (window probe) this is designed to prevent deadlock
- Senders may delay sending data, e.g., instead of sending the 2kiB immediately, could wait for a further 2kiB to fill the 4kiB receive window

© 2025 University of Melbourne

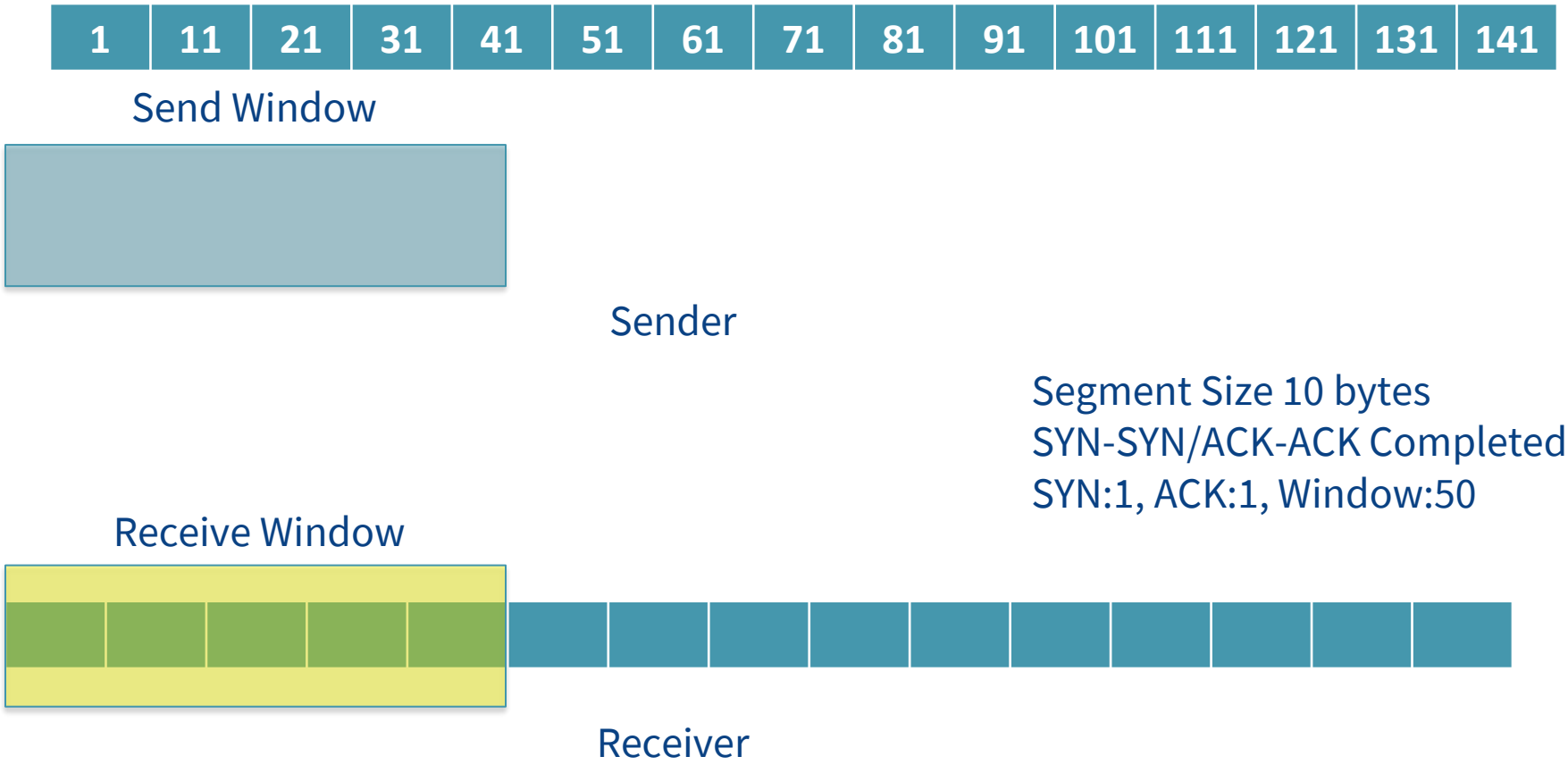
TCP Sliding Window

- Send window
  - What data the sender is able to send – unacknowledged segments and unsent data that will fit into the receive window
- Receive window
  - Amount of data the receiver is willing to receive – window size in ACK

- Other windows are maintained for congestion control

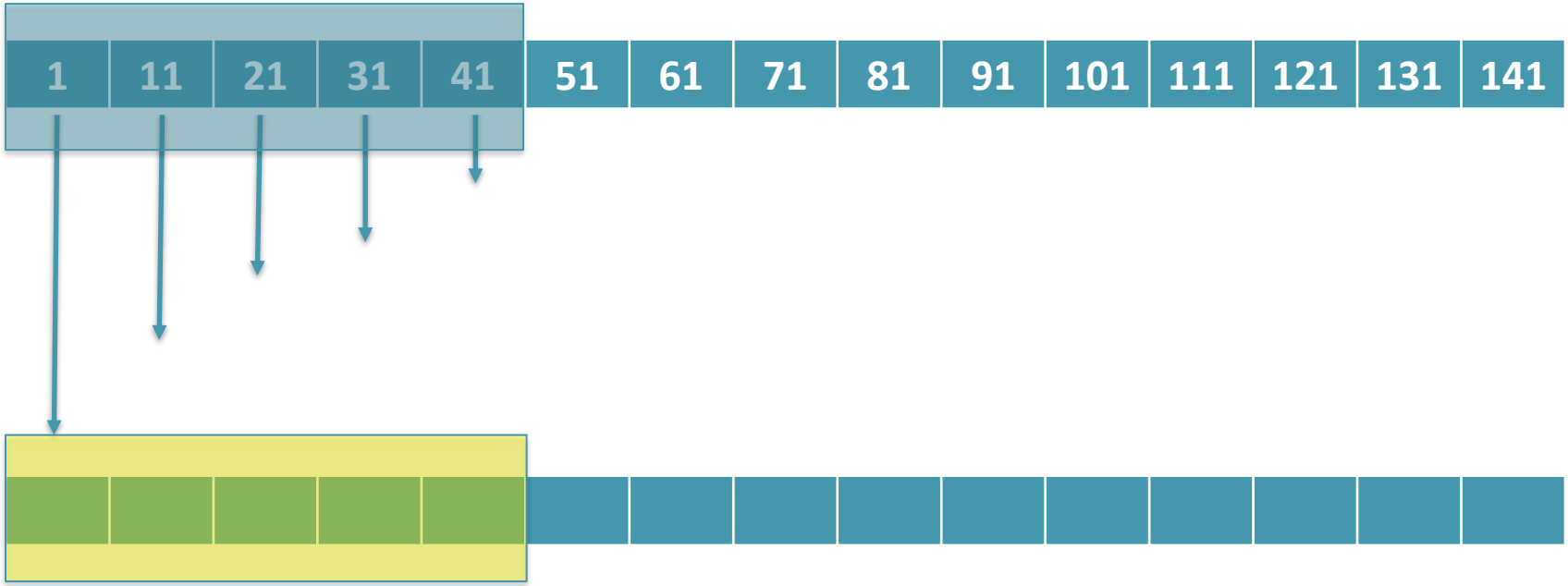
© 2025 University of Melbourne

TCP Sliding Window



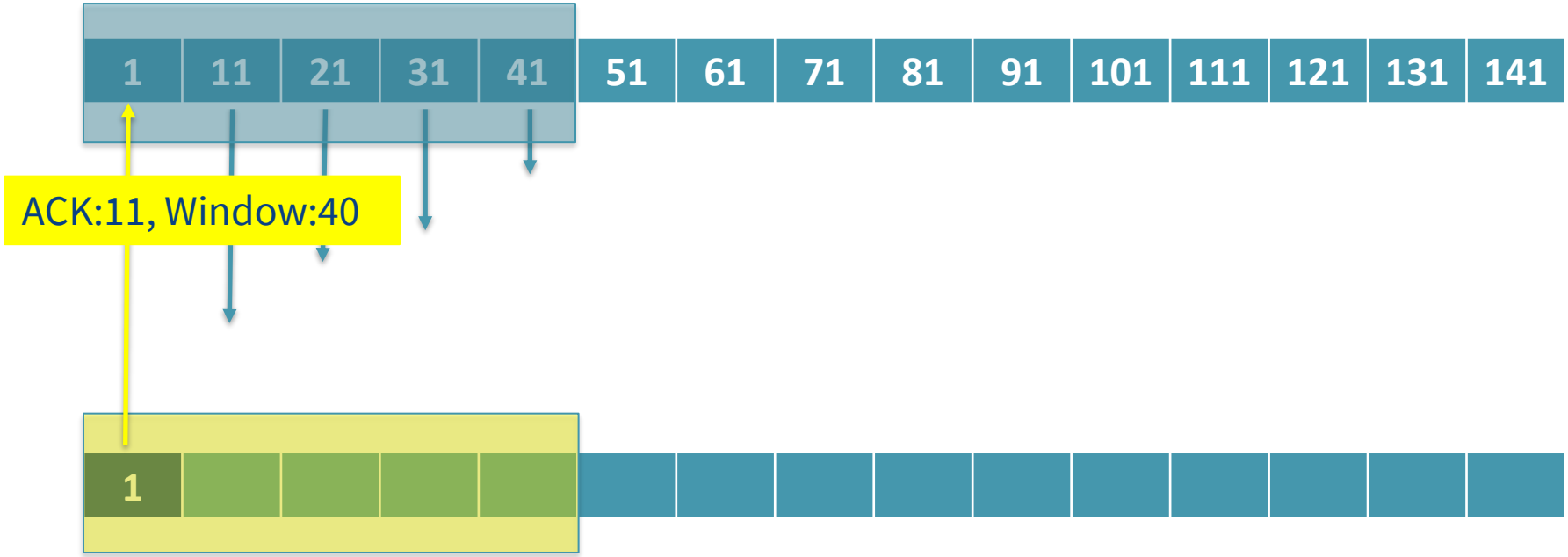
© 2025 University of Melbourne

TCP Sliding Window

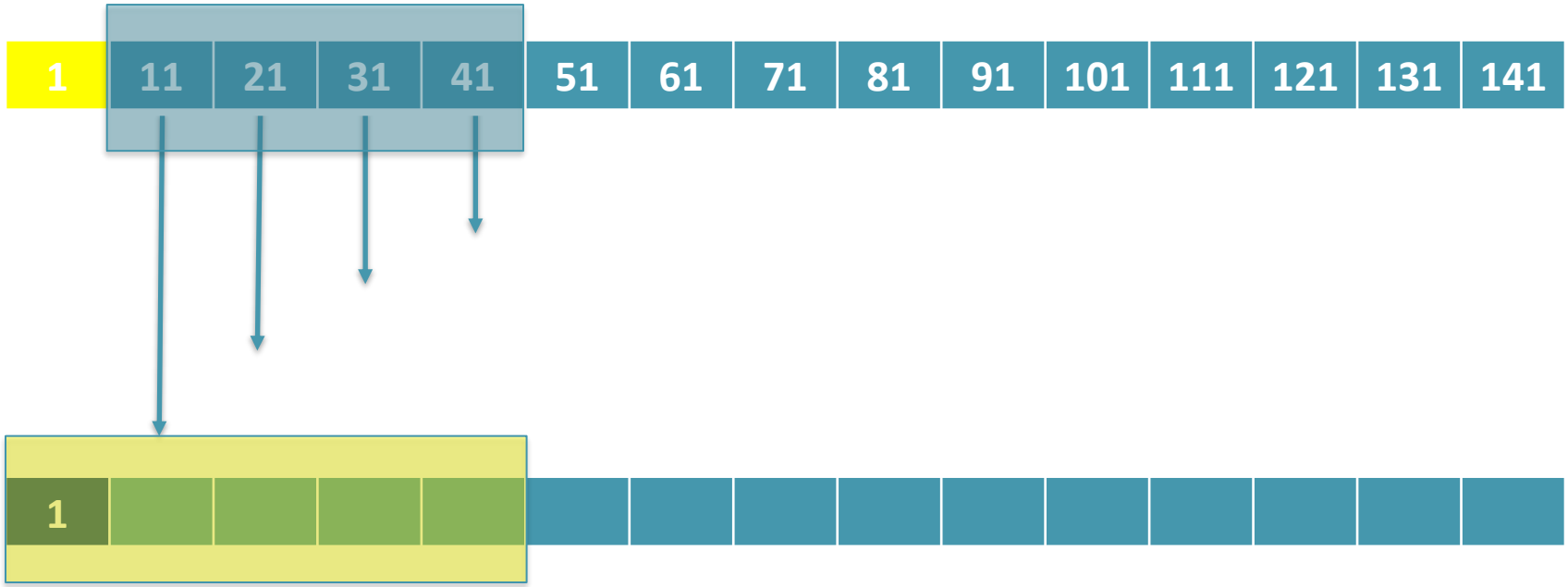


© 2025 University of Melbourne

TCP Sliding Window

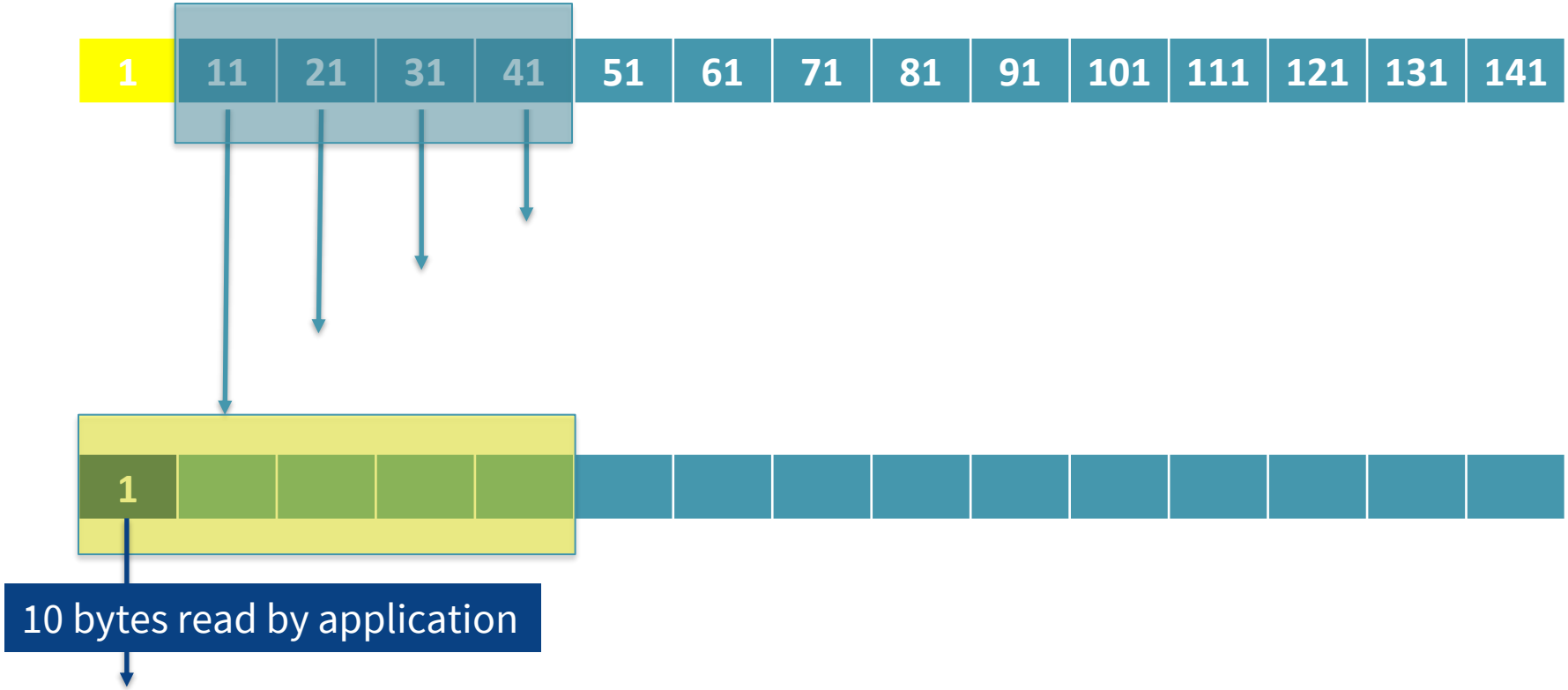


TCP Sliding Window



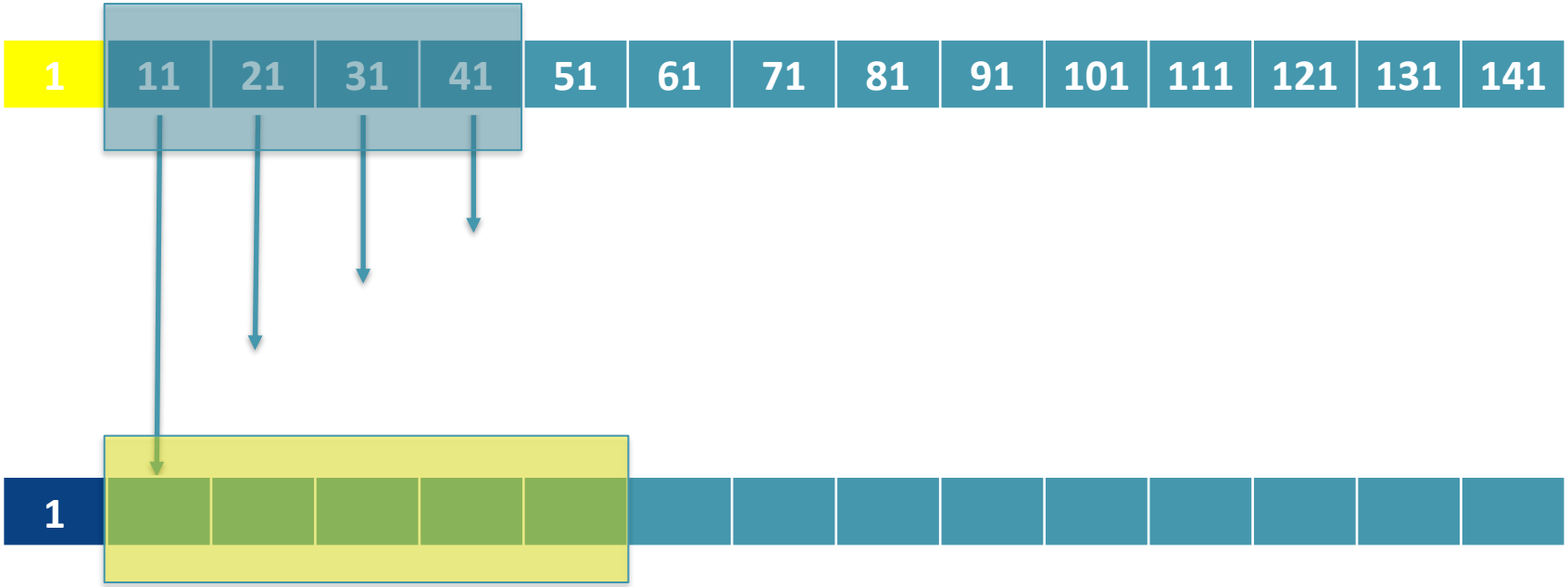
© 2025 University of Melbourne

TCP Sliding Window



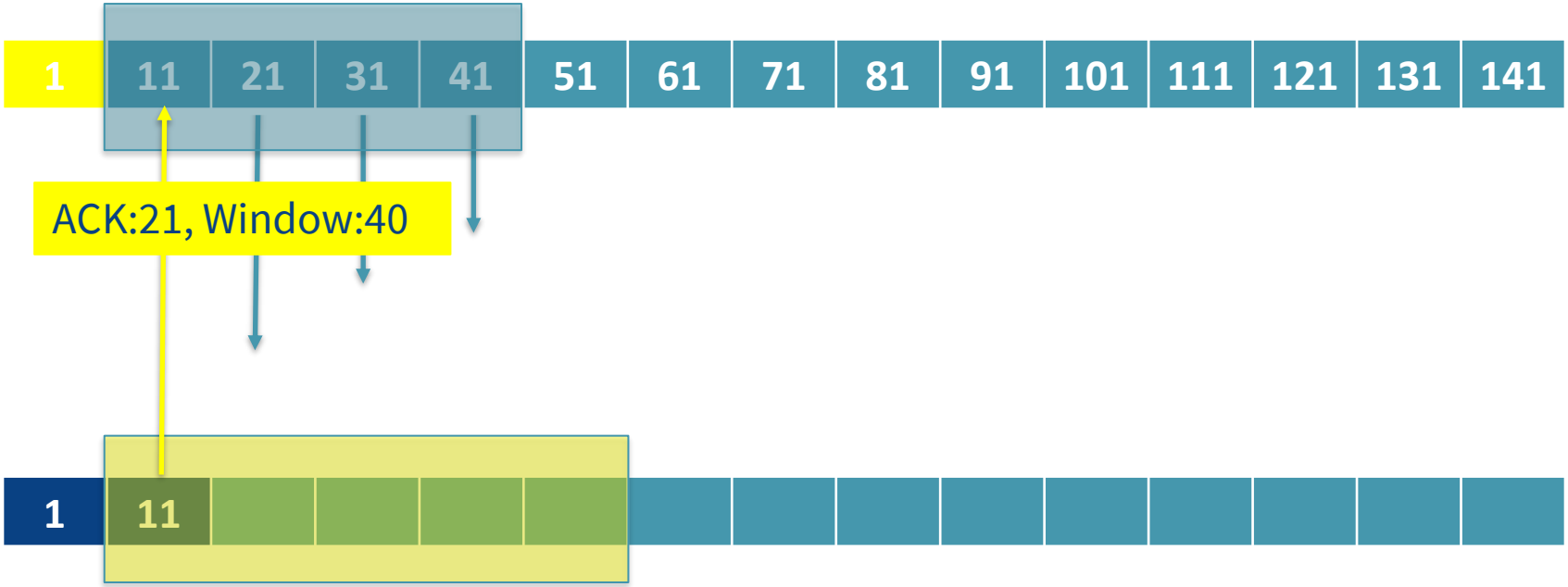
© 2025 University of Melbourne

TCP Sliding Window



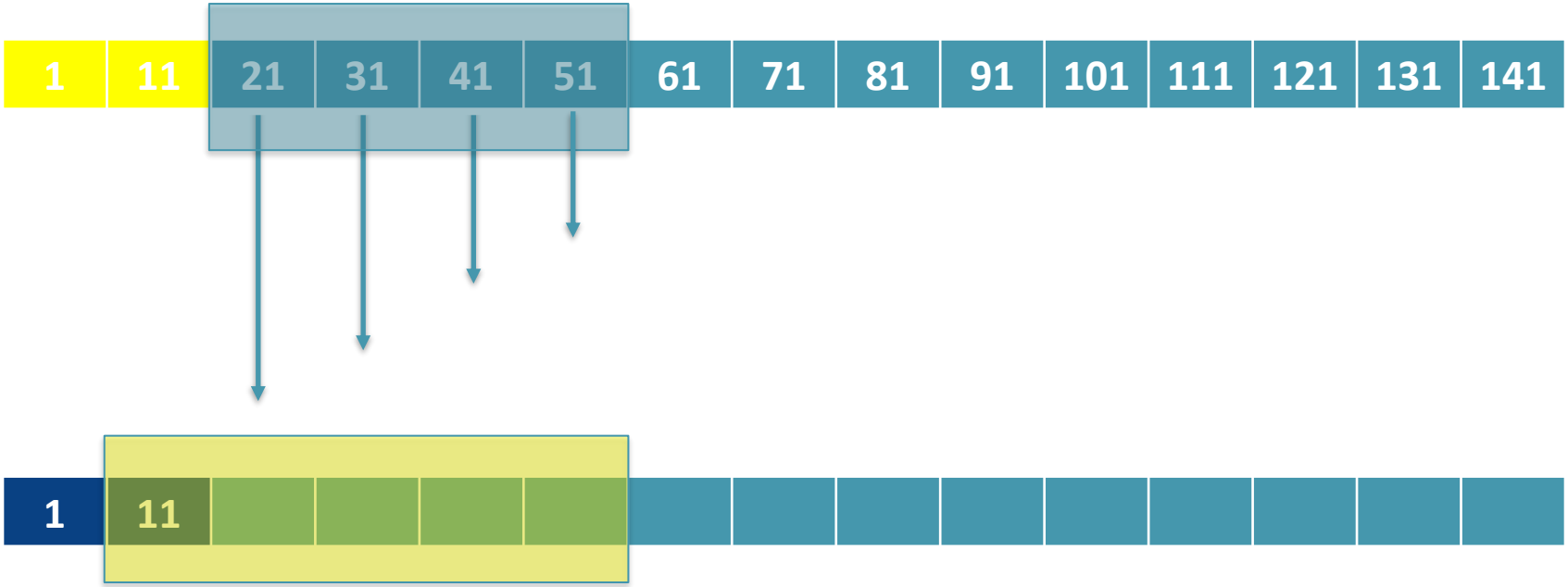
© 2025 University of Melbourne

TCP Sliding Window



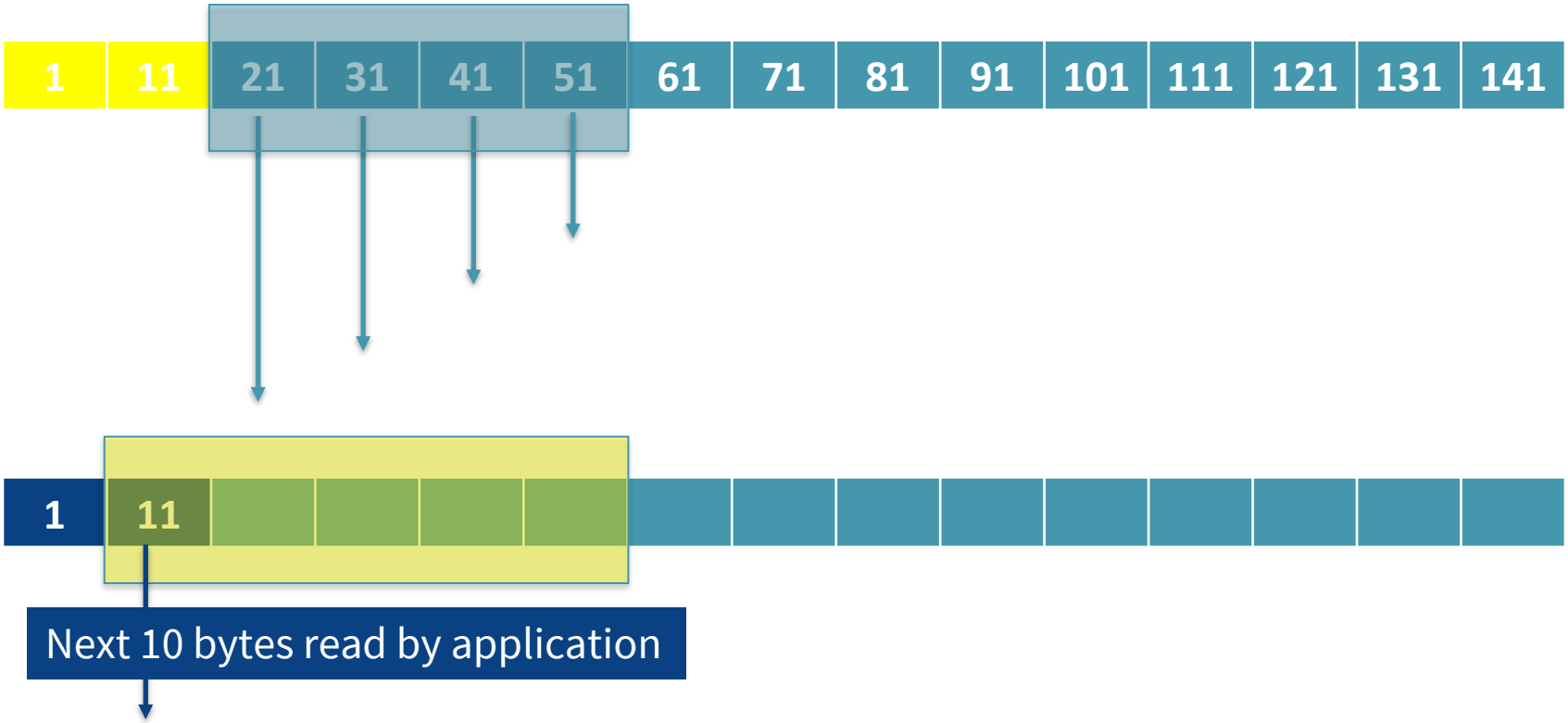
© 2025 University of Melbourne

TCP Sliding Window



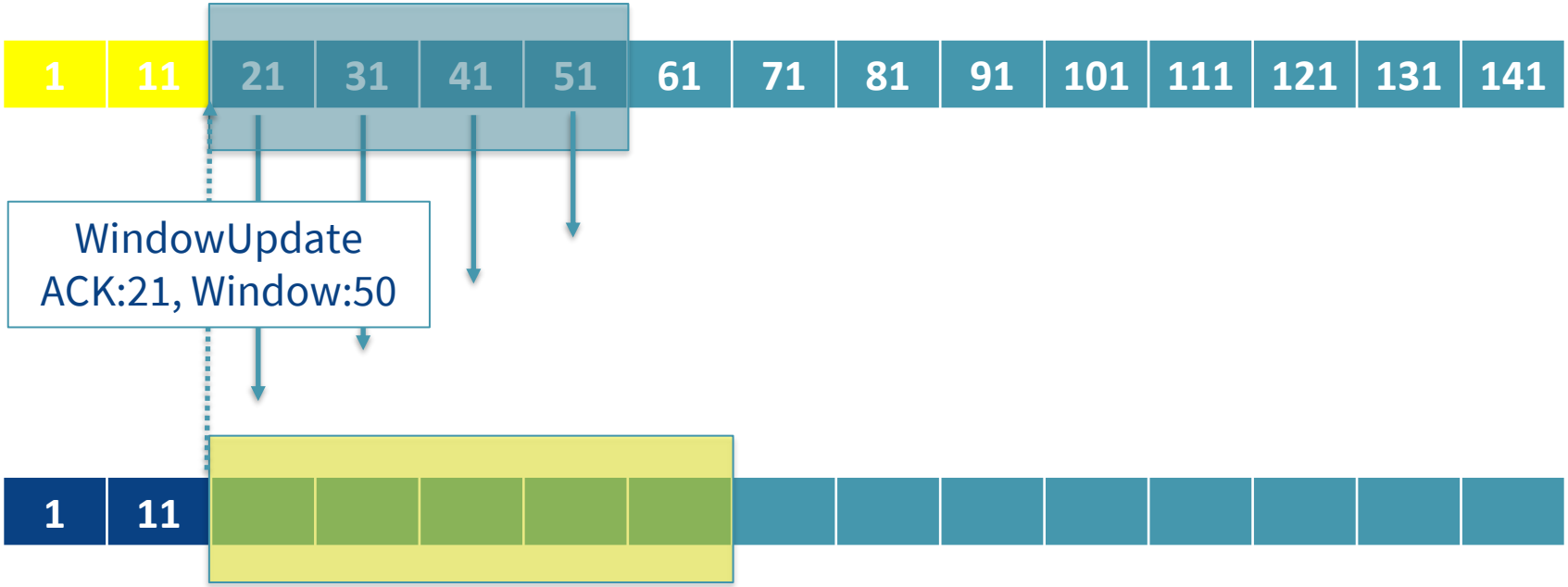
© 2025 University of Melbourne

TCP Sliding Window



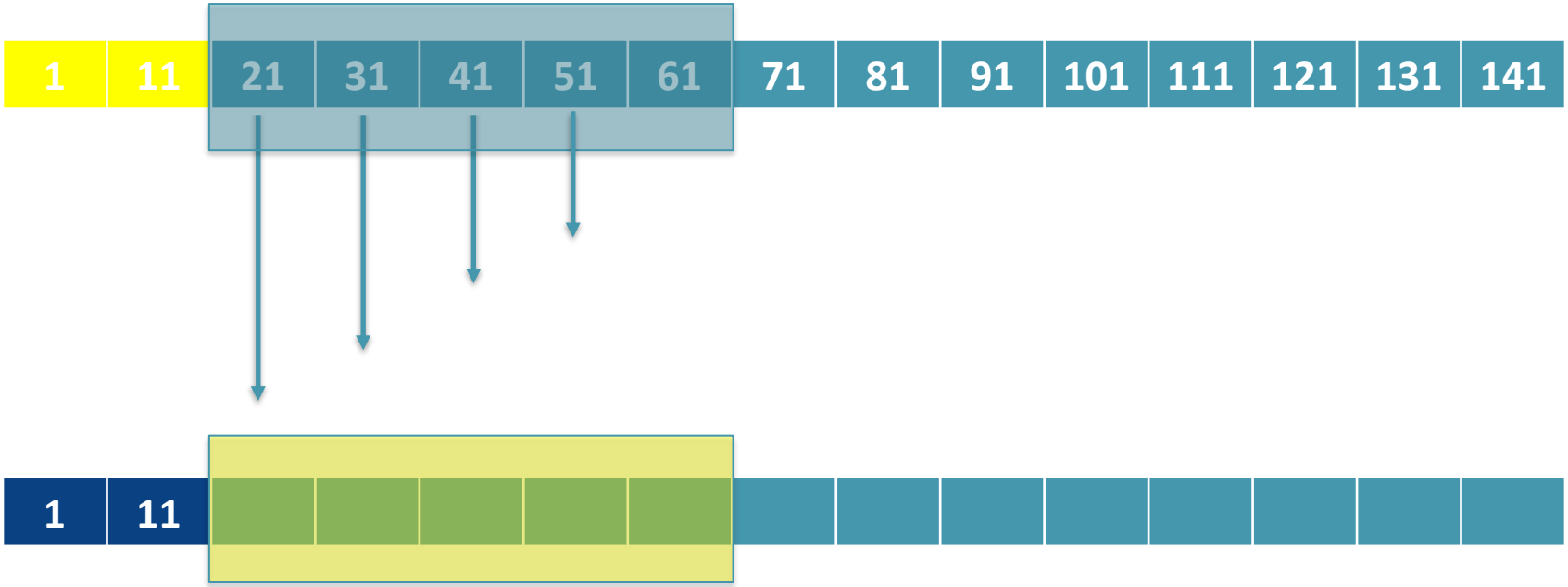
© 2025 University of Melbourne

TCP Sliding Window – Window Update



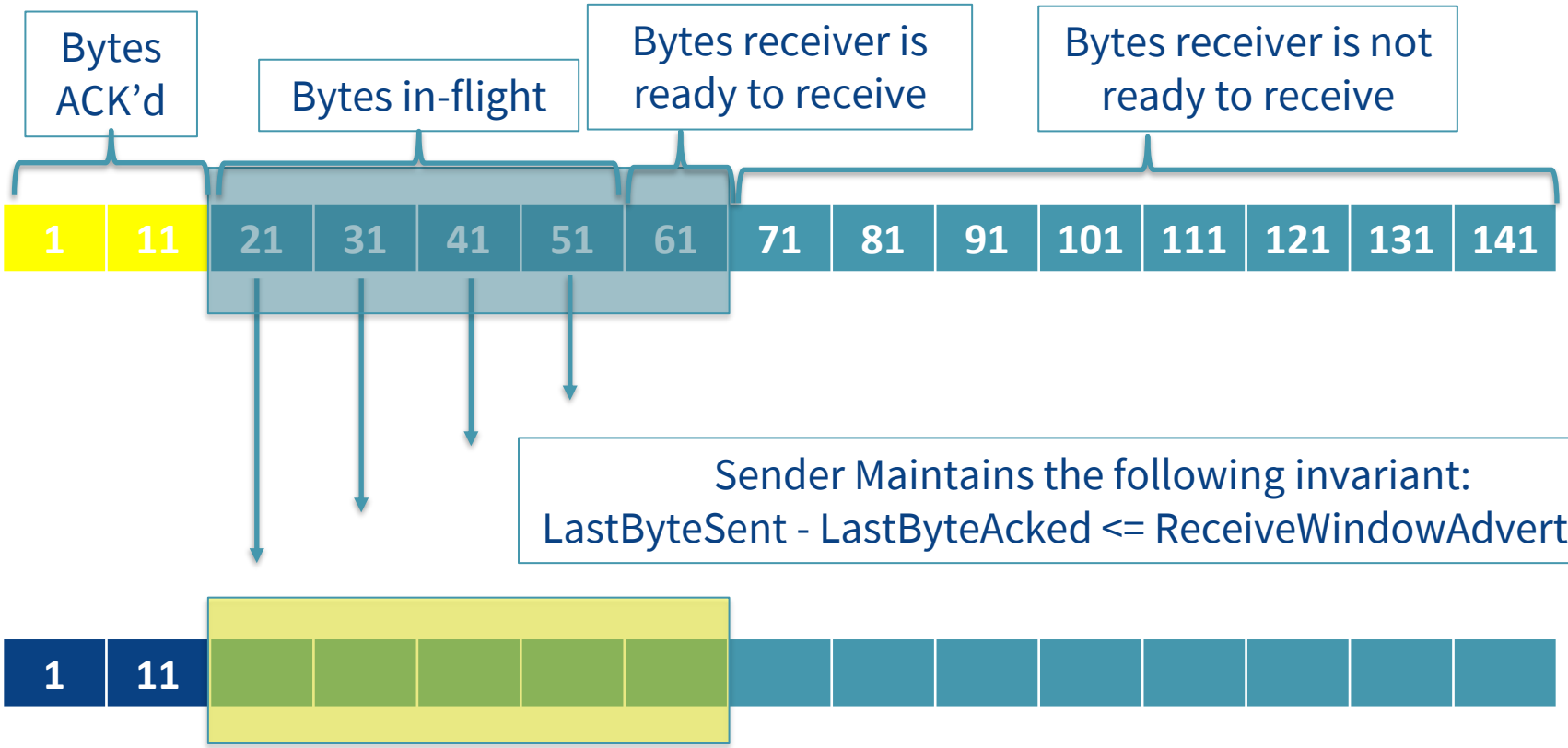
© 2025 University of Melbourne

TCP Sliding Window



© 2025 University of Melbourne

TCP Sliding Window



© 2025 University of Melbourne